游戏埋点接入文档

概述

本文档可应用于游戏行为埋点的对接。 由游戏客户端、服务端发送http 请求进行游戏行为埋点上报。

接入流程

1. 创建项目

接入前请于 `dmp` 后台生成 项目别名 及校验 `secret` 用于项目接入时签名。

2. 接口地址

地区	网关地址
国内	https://dcapi.zsdatac.com/v1/dc_api
海外	

3. 接口签名

签名算法采用 `HMAC-SHA256` ，并引入 nonce 防重放机制。客户端需在每次请求时按下述规则生成签名并在请求头中携带。

必需请求头

- `X-Project-Alias` : 项目别名
- `X-Timestamp` : 时间戳（秒或毫秒）
- `X-Nonce` : 随机字符串（建议 16–32 字符，确保全局唯一）
- `X-Sign` : 签名字符串（小写 hex）

校验规则

- 时间戳允许偏差：±5 分钟
- Nonce 有效期：10 分钟；同一 `(project_alias, timestamp, nonce)` 在有效期内不可重复
- 签名算法：HMAC-SHA256（key 为项目 `secret`）

签名串规范

1. 取参与签名的字段数组：`[project_alias, timestamp, nonce]`
2. 将数组元素按字典序升序排序
3. 使用换行符 `\n` 进行拼接，得到 `message`
4. 使用项目 `secret` 作为 HMAC key，对 `message` 做 HMAC-SHA256
5. 将结果编码为小写十六进制，即为 `X-Sign`

> 注意
>
> timestamp 支持秒或毫秒（毫秒会在服务端自动除以 1000）
>
> 推荐 `nonce` 使用高熵的随机值（比如 UUIDv4 或 128-bit 随机），避免可预测

请求示例

- Golang

代码块

```go
func GenHMACSHA256(secret string, parts []string) string {
    cp := append([]string(nil), parts...)
    // 字典序排序
    sort.Strings(cp)
    // 以\n拼接，避免字段粘连歧义
    msg := strings.Join(cp, "\n")
    mac := hmac.New(sha256.New, []byte(secret))
    mac.Write([]byte(msg))
    return hex.EncodeToString(mac.Sum(nil))
}
```

- Node

代码块

```node
function genHmacSha256(secret, parts) {
  const sorted = [...parts].sort();
  const msg = sorted.join('\n');
  const mac = crypto.createHmac('sha256', secret).update(msg).digest('hex');
  return mac.toLowerCase();
```

```
}
```

4. 日志格式

事件定义

埋点事件触发时， 由统一的结构进行上报。 事件定义主要包含以下内容： `谁，在哪？什么时间？做了什么事情？`

事件由： 系统属性（如 事件标识、事件时间、用户标识等）、公共属性（渠道信息、广告信息）、自定义属性（ecpm、广告类型、游戏等级）组成

系统属性

系统属性包含： 时间、事件标识、自定义属性标识等字段组成

> 系统属性不可更改，比如 事件标识字段 event_id 为系统预定义，游戏上报时不可重名

字段标识	字段名称	字段类型	是否必传	描述
event_id	事件标识	string	是	
event_name	事件名称	string	否	传入后方便运营、产品后台分析数据。否则需求安排业务同学手动更新
event_time	事件触发时间	Int	是	
openid	用户标识	string	是	游戏账户
unionid	平台用户标识	String	否	平台用户
device_id	设备标识	string	否	设备标识（用于广告归因）

公共属性

公共属性由日常报表中常用的一些筛选维度组成。 如下列表所示

字段标识	字段名称	字段类型	描述	备注

ad_platform	广告平台	string	`gdt` - 广告点通 `adq` - 微信ADQ `mp` - 微信MP `toutiao` - 头条	新增广告平台时可通知更新文档
platform	平台	string	101 - 微信小游戏 102 - 抖音小游戏 103 - oppo 小游戏 104 - vivo 小游戏 105 - 手Q小游戏 106 - 快手小游戏 201 - Google 202 - AppStore 203 - APK	原字段为 channel 为方便与 渠道号等区别改为 platform
package_name	游戏包名	String		
channel_id	渠道号	string	渠道号为应用最小统计维度， 小游戏可传 广告来源 source	
ad_id	广告ID	string	广告ID	各广告平台定义不一致，统一内部定义
ad_account_id	广告账户	string	广告账户	
ad_group_id	广告组ID	string	广告组ID	
scene	微信场景值	string		
os	系统	string		
os_ver	系统版本	string		
app_ver	游戏版本	string		
cfg_version	配置版本	String		
ab_test	AB测试	string		

自定义属性

自定义属性主要用于告知用户做了啥，比如闯关等级、广告ecpm 等

> *注意：事件属性均放于 `properties` 对象下

代码块

```
{
  "event_id": "sys.ad",
  "event_time": 1657092829,
  "event_name": "广告",
  "properties": {
    "ecpm": 6.8
  }
```

事件类型

预定义事件

系统会根据部分预定义的事件做数据清洗。 因而定义部分事件为预定义事件， 预定义事件为项目必须接入。

`DMP` 后台会根据预定义事件搭建基础报表模版。

事件标识	事件名称	触发时机
sys.activate	激活	用户首次进入游戏时触发（客户端）
sys.register	注册	服务端注册成功时触发（服务端）
sys.login	登录	用户登录游戏时触发（服务端）
sys.ad	广告点击	用户点击广告时触发（客户端）
sys.online	用户时长	用户定义触发（客户端）
sys.exception	客户端异常	游戏异常时触发上报（客户端）
sys.userset	用户属性更新	
sys.adconsume	广告消耗事件	脚本拉取上报

自定义事件

为游戏内行为事件埋点。 自定义事件标识 `event_id` 可由英文字母加下划组成。 如 `game_start` - 开始游戏

> 为方便区分， 自定义事件标识不得添加 `sys.` 前缀

5. 用户ID体系

用户ID体系主要用于用户归因。 包含： `device_id` `openid` `unionid`

`device_id` 用于归因设备，识别用户是否在同一台设备上登录。

> 小游戏device_id 可由客户端生成随机串存储到本地缓存中。

`openid` 为游戏账户（非角色账户）、SDK账户， 一般由 Hash(appid + uid）生成。

> 暂时不考虑角色账户，如游戏有创角流程， 在自定义属性 `properties` 上报即可。

`unionid` 为平台用户ID Hash(公司主体 + uid) 值， 用于标识平台用户。 如微信小游戏应用绑定开放平台后， 即可获取unionid。

6. 预定义事件接入

以下表格列举的字段标识均为必传字段，并非所有事件属性。 公共属性都为可选埋点属性，没一一列举出来。

激活(sys.activate)

用户第一次打开游戏时触发上报。用户删除应用或清空小游戏缓存进入游戏，可计为多次激活

> 触发时机由游戏（SDK）定义

属性标识	字段类型	描述	示例
device_id	string	设备标识	
package_name	string	包名	com.seegames.01
event_time	int	激活时间	

注册(sys.register)

用户登录游戏第一次创建账户时触发上报。

属性标识	字段类型	描述	示例
device_id	string	设备标识	
openid	string	用户标识	oBtG85JTppIlDaoloqB8BomFMpwQ
unionid	string	平台用户标识	
event_time	int	时间	
channel_id	string	渠道信息	微信小游戏可采用 source 字段值
properties.share_openid	string	分享用户标识	
properties.ip	string	IP	
properties.network	string	网络类型	

登录(sys.login)

用户每次登录游戏或用户注册时触发上报，

属性标识	字段类型	描述	示例
openid	string	用户标识	oBtG85JTppIlDaoloqB8BomFMpwQ
unionid	string	平台用户标识	
event_time	int	时间	
device_id	string	设备标识	
channel_id	string	渠道信息	微信小游戏可采用 source 字段值
package_name	string	包名	

在线时长(sys.online)

用户在线时长统计方案

- 方案1

用户退出游戏时触发上报

> 用户切出游戏时无法获取当次时长

- 方案2（推荐）

客户端采用N分钟一次心跳

> 仅丢失最后一次心跳时长

属性标识	字段类型	描述	示例
openid	string	用户标识	oBtG85JTppIlDaoloqB8BomFMpwQ
unionid	string	平台用户标识	
event_time	int	时间	
device_id	string	设备标识	
channel_id	string	渠道信息	微信小游戏可采用 source 字段值
package_name	string	包名	
properties.duration	int	时长(秒)	

广告(sys.ad)

用户点击、请求、加载广告等事件合并成一个事件， 可添加自定义属性类型进行区分。

属性标识	字段类型	描述	示例
openid	string	用户标识	oBtG85JTppIlDaoloqB8BomFMpwQ
unionid	string	平台用户标识	
event_time	int	时间	
device_id	string	设备标识	
channel	string	渠道信息	微信小游戏可采用 source 字段值
properties.ad_id	string	广告标识	57239047693
properties.placement	string	广告位类型	rewarded - 激励视频 banner - banner 广告 interstitial - 插屏广告

properties.action_type	string	行为类型	start - 点击播放 load - 请求成功 load_fail - 请求失败 cancel - 中途取消 end - 完成
properties.ad_duration	double	时长	

系统异常(sys.exception)

属性标识	字段类型	描述	示例
openid	string	用户标识	oBtG85JTppIlDaoloqB8BomFMpwQ
properties.content	string	异常信息	
properties.error_type	string	异常类型	系统崩溃、程序异常等

用户属性变动 (sys.userset)

用户属性变动事件主要用于终端清洗用户数据。系统会以第一次接收到的数据类型为准创建表字段。因而上报的字段需要明确定义。

- 时间类型用户属性

比如 `last_login_time` - 最近登录时间，上报时必须为时间类型字段如 `2025-01-01 00:00:00` 。 否则判断成其它类型时，后台时间筛选组件不可用。

- 累加类型用户属性

累加类型的用户属性必须为数值类型。否则会计算失败。

- 首次类型的用户属性

`首次` 用户属性尽量在 `sys.register` 中进行上报。 新增的首次用户属性可以后续的事件中上报

属性标识	字段类型	描述	示例
openid	string	用户标识	oBtG85JTppIlDaoloqB8BomFMpwQ

event_type	string	user_set - 赋值 user_setonce - 首次	
properties.first_login_time	string	首次登录	

> 注意：
>
> 以上标识中包含properties的标识均为对象属性，如下所示
>
> 代码块
>
> ```
> 1  {
> 2      "properties": {
> 3          "first_login_time": "2025-01-01 00:00:00"
> 4      }
> 5  }
> ```

退出(sys.exit)

广告消耗

广告消耗数据为动态更新数据， 由采集接口导入事件表后， 会由后端数据作业脚本清洗成广告消耗表

属性标识	字段类型	描述	示例
channel_id	string	渠道号	
ad_id	string	广告ID	
ad_platform	string	广告平台	
platform	string	平台	
ad_account_id	string	广告账户	
properties.cost	double	消耗	
properties.real_cost	double	实际消耗	

7. 自定义事件接入

自定义事件接入时只需要包含系统属性， 公共属性最好能跟预定义一致。否则会出现两类型事件归因不一致问题。 接入格式参考如下所示

8. 事件格式参考

单用户多事件

单用户多事件可用于 sdk 客户端批量事件上报， 主要用途是降低客户端的请求。

代码块

```
{
  "openid": "11002",
  "channel_id": "10401",
  "device_id": "4a58162c-3b92-4f7a-b987-4bfddb904ac2",
  "events": [
    {
      "event_id": "sys.ad",
      "event_time": 1657092829,
      "event_name": "广告",
      "properties": {
        "order_id": "no12312412",
        "ecpm": 6.8
      }
    },
    {
      "event_id": "sys.activate",
      "event_name": "激活",
      "event_time": 1657092829,
      "properties": {
      }
    }
  ]
}
```

多用户多事件

多用户多事件用于服务端批量上报， 减少服务端请求。

代码块

```
[
  {
    "openid": "11002",
    "channel_id": "10401",
    "device_id": "4a58162c-3b92-4f7a-b987-4bfddb904ac2",
    "events": [
```

```
7         {
8           "event_id": "sys.activate",
9           "event_name": "激活",
10          "event_time": 1657092829,
11          "properties": {
12            "xxx": "xxxx",
13            "abc": 111
14          }
15        }
16      ]
17    },
18    {
19      "openid": "11003",
20      "channel_id": "10401",
21      "device_id": "4a58162c-3b92-4f7a-b987-4bfddb904ac2",
22      "events": [
23        {
24          "event_id": "sys.ad",
25          "event_time": 1657092829,
26          "event_name": "广告",
27          "properties": {
28            "xxx": "xxxx",
29            "xxx": 111
30          }
31        },
32        {
33          "event_id": "sys.activate",
34          "event_name": "激活",
35          "event_time": 1657092829,
36          "properties": {
37            "xxx": "xxxx",
38            "xxx": 111
39          }
40        }
41      ]
42    }
43  ]
```

单事件

支持单事件上报。 为减少网络开销，数据上报时，可尽量用 `单用户事件` 及 `多用户事件` 结构进行上报。

代码块

```
{
  "openid": "11002",
  "channel_id": "10401",
  "device_id": "4a58162c-3b92-4f7a-b987-4bfddb904ac2",
  "event_id": "sys.ad",
  "event_time": 1657092829,
  "event_name": "广告",
  "properties": {
    "order_id": "no12312412",
    "ecpm": 6.8
  }
}
```

dc_api_test.html

> 可下载该网页可用于联调测试